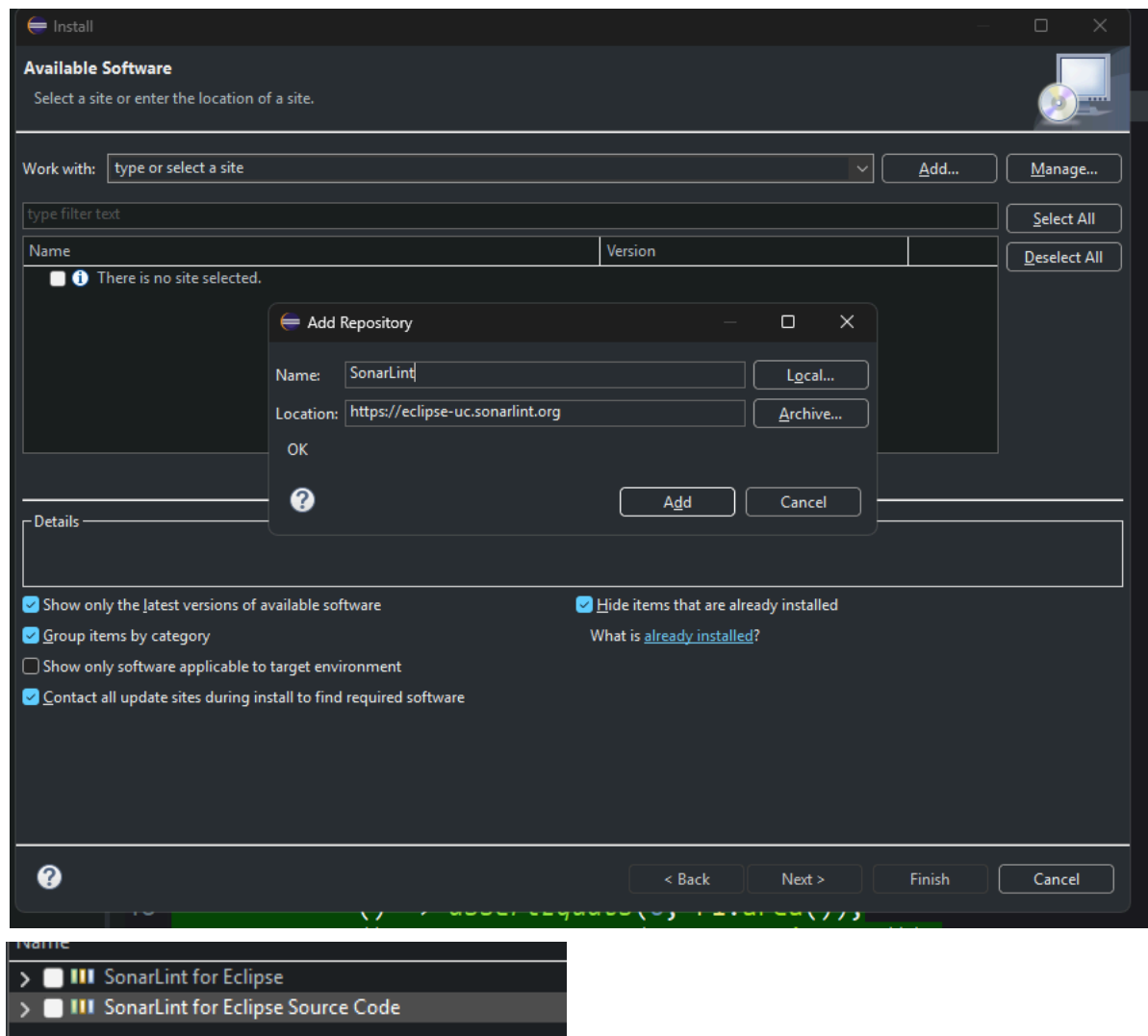


Ejercicio 3 – Analizador de código: SonarLint

Instalación y comprobación de SonarLint

Para realizar esta práctica se utilizó SonarLint, un complemento de análisis estático de código que se integra en Eclipse y permite detectar problemas de calidad, errores potenciales y malas prácticas de programación mientras se desarrolla una aplicación.

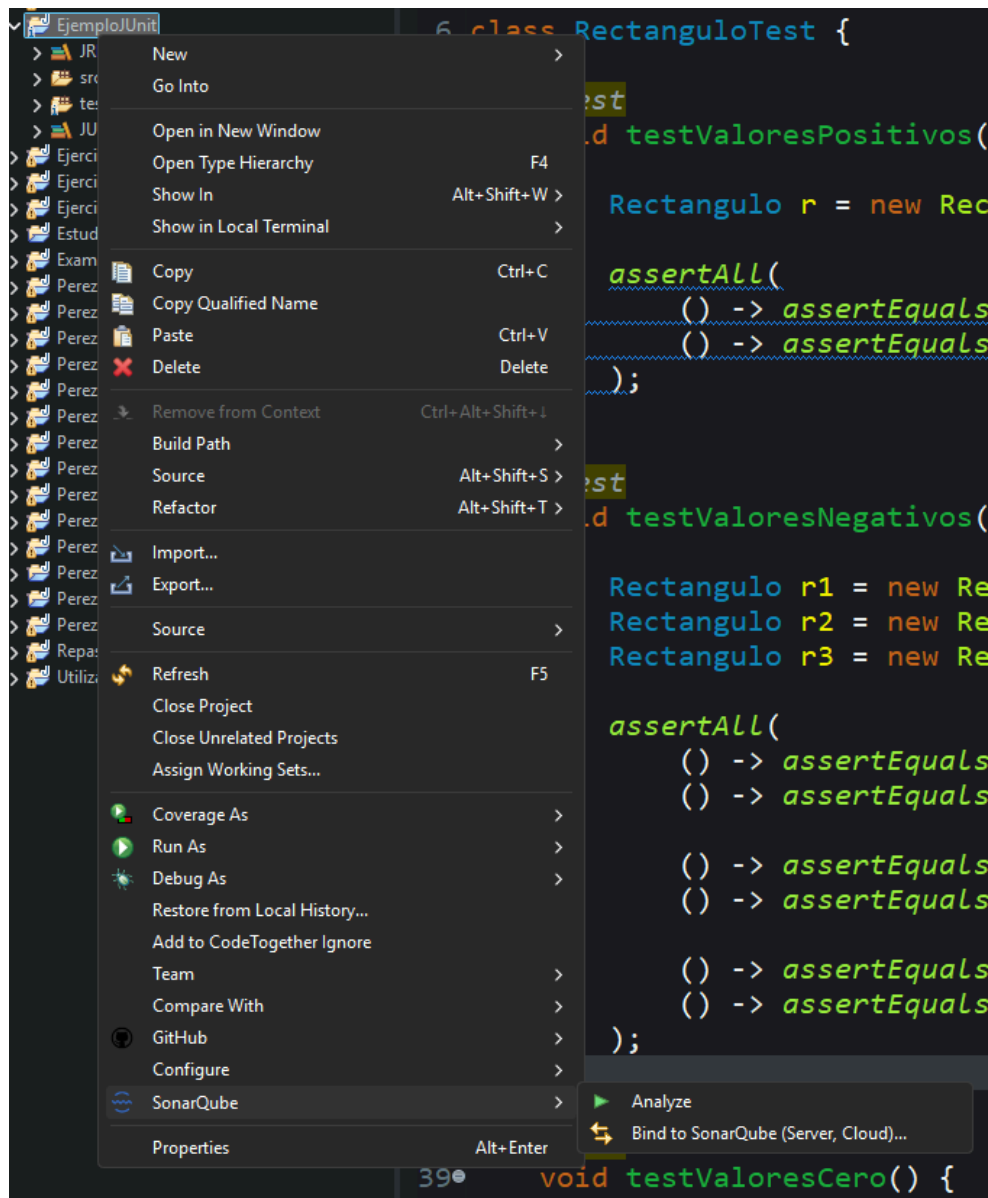
Antes de comenzar el análisis, se comprobó que SonarLint estaba correctamente instalado en Eclipse mediante la ventana **Installation Details**, donde aparece el complemento **SonarLint for Eclipse** entre los componentes instalados.



Ejecución del análisis

Una vez comprobada la instalación, se abrió el proyecto desarrollado en el ejercicio 1 y se ejecutó el análisis utilizando la opción **SonarQube** → **Analyze** disponible en el menú contextual del proyecto.

Esta opción permite realizar un análisis automático del código fuente utilizando las reglas de calidad definidas por SonarLint.

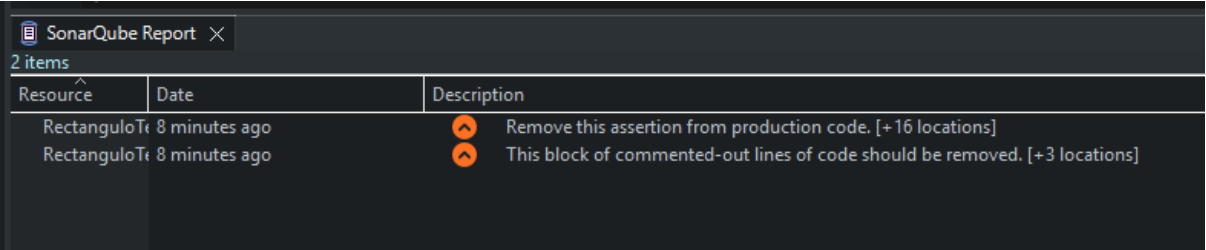


Resultados obtenidos

Tras ejecutar el análisis, SonarLint generó un informe con las incidencias detectadas en el proyecto.

Entre los problemas encontrados destacan la presencia de aserciones en ubicaciones no recomendadas y la existencia de bloques de código comentado que deberían eliminarse para mejorar la calidad y el mantenimiento del código.

Estos resultados muestran cómo SonarLint es capaz de identificar automáticamente posibles problemas de programación y proporcionar recomendaciones para corregirlos antes de que la aplicación sea puesta en producción.



The screenshot shows a 'SonarQube Report' window with a table of 2 items. The table has columns for Resource, Date, and Description. The first item is 'RectánguloT' with a date of '8 minutes ago' and a description 'Remove this assertion from production code. [+ 16 locations]'. The second item is 'RectánguloT' with a date of '8 minutes ago' and a description 'This block of commented-out lines of code should be removed. [+ 3 locations]'. Both items have an orange icon with a white arrow pointing up.

Resource	Date	Description
RectánguloT	8 minutes ago	Remove this assertion from production code. [+ 16 locations]
RectánguloT	8 minutes ago	This block of commented-out lines of code should be removed. [+ 3 locations]

Relación entre SonarLint y SonarQube

SonarLint y SonarQube son herramientas desarrolladas por la misma organización y comparten el mismo objetivo: mejorar la calidad y seguridad del código fuente mediante análisis estáticos.

Ambas herramientas utilizan reglas similares para detectar errores, vulnerabilidades de seguridad, duplicidades de código y problemas de mantenimiento. Sin embargo, están orientadas a escenarios diferentes y se complementan entre sí.

SonarLint trabaja directamente dentro del entorno de desarrollo, proporcionando información inmediata al programador mientras escribe el código. Por el contrario, SonarQube es una plataforma centralizada que analiza proyectos completos y almacena los resultados para que puedan ser consultados por todo el equipo de desarrollo.

Además, SonarLint puede conectarse a un servidor SonarQube para utilizar las mismas reglas de calidad definidas por la organización, garantizando que todos los desarrolladores trabajen bajo los mismos criterios.

Similitudes y diferencias

Similitudes

- ❖ Ambas herramientas realizan análisis estáticos de código.
- ❖ Detectan errores, vulnerabilidades y problemas de calidad.
- ❖ Utilizan reglas de análisis similares.
- ❖ Tienen como objetivo mejorar la mantenibilidad y seguridad de las aplicaciones.

Diferencias

SonarLint

- Funciona dentro del IDE.
- Analiza el código mientras se programa.
- Uso principalmente individual.
- No almacena históricos.
- Gratuito y ligero.

SonarQube

- Funciona como plataforma independiente.
- Analiza proyectos completos.
- Uso orientado a equipos de desarrollo.
- Mantiene métricas e históricos de los análisis.
- Dispone de versiones gratuitas y comerciales.

Conclusión

Mediante esta práctica se ha comprobado el funcionamiento de SonarLint como herramienta de análisis estático de código integrada en Eclipse.

El análisis realizado sobre el proyecto desarrollado en el ejercicio anterior ha permitido detectar incidencias relacionadas con la calidad del código, demostrando la utilidad de la herramienta para identificar problemas durante el proceso de desarrollo.

Asimismo, se ha estudiado la relación existente entre SonarLint y SonarQube, comprobando que ambas herramientas se complementan para mejorar la calidad del software tanto a nivel individual como en entornos de desarrollo colaborativos.

